

Iris MLOps Deployment — Scenario 1 to 11

Flask + Waitress + Nginx: A Complete Deployment Walkthrough

Deployment Study Guide

Contents

Project Overview	3
Folder Structure (project name: <code>mlops2</code>)	3
What This Project Does	3
requirements.txt (inferred from imports)	3
Scenario 1 — Train the Model + Run app.py	4
Step 1 — Get the CSV	4
Step 2 — <code>train_model.py</code> (full content)	4
Step 3 — Run Training	5
Step 4 — <code>app.py</code> , Scenario 1 version (Flask's own dev server)	5
Step 5 — <code>templates/index.html</code>	6
Step 6 — <code>templates/result.html</code>	7
Step 7 — Run and Test	8
Where Does Nginx Live? (Not Used Yet)	8
Scenario 2 — Switch to Waitress	9
Before (Scenario 1 — Flask's own dev server)	9
After (Scenario 2 — Waitress)	9
What Is a WSGI Server?	9
Nothing Else Changes	9
Access	9
host="0.0.0.0" vs 127.0.0.1 — Not the Same Thing	10
Scenario 3 — Nginx, 1 Server Block, 1 Domain, 1 app.py	11
Step 1 — <code>hosts</code> file	11
Step 2 — <code>nginx.conf</code>	11
What Each <code>proxy_set_header</code> Line Does	11
Starting and Stopping Nginx	11
Full Default <code>nginx.conf</code> (Before Editing)	12
Every Line Explained	12
What Did NOT Change	13
Access	13
Scenario 4 — Nginx, 1 Server Block, 2 Domains, 1 app.py (Virtual Hosting)	14
Step 1 — <code>hosts</code> file (both domains)	14
Step 2 — <code>nginx.conf</code> (one block, both names)	14
Step 3 — <code>app.py</code> must check <code>request.host</code>	14
What Domains and Subdomains Actually Are	14
Access	15

Scenario 5 — Nginx, 2 Server Blocks, 2 Domains, Still 1 app.py	16
Before (Scenario 4 — 1 block)	16
After (Scenario 5 — 2 blocks)	16
Why Split If the Result Is Identical?	16
Access	16
Scenario 6 — Nginx, 2 Server Blocks, 2 Waitress Instances, 2 app.py	17
Change 1 — app.py (remove the host check)	17
Change 2 — new file app_api.py	17
Change 3 — nginx.conf (different ports per block)	17
Run Both Apps	18
Access	18
Scenario 7 — 4 Physical Machines, Same Wi-Fi/LAN	19
Step 1 — Find Each Machine’s LAN IP	19
Change 1 — nginx.conf on laptop-2 (real IPs, not 127.0.0.1)	19
Change 2 — hosts file moves to laptop-1 (the client)	19
Change 3 — 0.0.0.0 binding now matters for real	19
Change 4 — Firewalls (new requirement)	19
Scenario 8 — HTTPS via OpenSSL (Self-Signed, Browser Warns)	21
Step 1 — Generate a Self-Signed Certificate	21
Step 2 — Add HTTPS Blocks	21
The HTTP -> HTTPS Redirect Design Issue	22
What listen 443 ssl Does	22
ssl_protocols and ssl_ciphers	22
Why the Browser Still Shows “Not Secure”	22
Getting a Publicly Trusted Certificate (Let’s Encrypt via Docker)	22
Scenario 9 — HTTPS via mkcert (No Warning)	24
Step 1 — Install mkcert	24
Step 2 — Create a Local Trusted Authority	24
Step 3 — Generate Certs for Both Domains	24
Step 4 — nginx.conf (swap the cert paths)	24
Important Limit	24
Scenario 10 — Serving Static Files via Nginx	26
Step 1 — Create a static/ Folder	26
Step 2 — Move CSS Out of the HTML Files	26
Step 3 — nginx.conf (add a /static/ block)	26
Why Web Servers Serve Static Files Faster	27
Important Note on Separate HTTP/HTTPS Blocks	27
Scenario 11 — Load Balancing and Sticky Sessions	28
Step 1 — Run Multiple Copies, Different Ports	28
Step 2 — nginx.conf (add upstream, change proxy_pass)	28
Default Behavior: Round-Robin	28
Other Load Balancing Algorithms	29
Sticky Sessions (Nginx Plus Only, Not in Free Nginx)	29
Appendix — Quick Reference	31
The Full Ladder, One Table	31
Ports Used Throughout	31
Files That Ever Change Across All 11 Scenarios	31

Project Overview

Folder Structure (project name: mlops2)

```
mlops2/
|-- app.py
|-- train_model.py
|-- requirements.txt
|-- templates/
|   |-- index.html
|   |-- result.html
|-- models/
|   |-- gaussian_nb_model.pkl      (created after training)
|-- data/
|   |-- Iris (1).csv               (place here before training)
```

What This Project Does

A Gaussian Naive Bayes classifier trained on the classic Iris dataset, served through a Flask web form. The user enters 4 flower measurements (sepal length/width, petal length/width) and gets back a predicted species.

requirements.txt (inferred from imports)

```
flask
waitress
pandas
scikit-learn
```

Install with:

```
pip install -r requirements.txt
```

Scenario 1 — Train the Model + Run app.py

Step 1 — Get the CSV

train_model.py reads from:

```
DATA_PATH = os.path.join("data", "Iris (1).csv")
```

Get the classic Iris dataset (e.g. Kaggle's "Iris Species" dataset, uciml/iris) and place it at `data/Iris (1).csv`.

Step 2 — train_model.py (full content)

```
import os
import pickle
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

DATA_PATH = os.path.join("data", "Iris (1).csv")
df = pd.read_csv(DATA_PATH)

rename_map = {
    "SepalLengthCm": "sepal_length",
    "SepalWidthCm": "sepal_width",
    "PetalLengthCm": "petal_length",
    "PetalWidthCm": "petal_width",
    "Species": "species",
}
df = df.rename(columns=rename_map)
if "Id" in df.columns:
    df = df.drop(columns=["Id"])

FEATURES = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
TARGET = "species"

X = df[FEATURES]
y = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)

print(f"Test Accuracy: {acc:.4f}")
print("\n Classification Report:\n")
print(classification_report(y_test, y_pred))

os.makedirs("models", exist_ok=True)
```

```

MODEL_PATH = os.path.join("models", "gaussian_nb_model.pkl")

with open(MODEL_PATH, "wb") as f:
    pickle.dump(model, f)

print(f"\nModel saved to: {MODEL_PATH}")

```

What it does, step by step:

1. Reads the CSV
2. Renames Kaggle's column names to simple lowercase names
3. Splits 80% train / 20% test, with `stratify=y` so both sets keep the same proportion of each species
4. Trains a `GaussianNB` model
5. Prints accuracy + classification report
6. Saves the trained model to `models/gaussian_nb_model.pkl`

stratify=y explained: without it, a random split could put mostly one species in train and few in test. With `stratify=y`, if the full dataset is 33/33/33 percent across 3 species, both train and test sets keep that same 33/33/33 split.

Step 3 — Run Training

```
python train_model.py
```

```

Test Accuracy: 0.9667
...classification report...
Model saved to: models/gaussian_nb_model.pkl

```

`os.makedirs("models", exist_ok=True)` creates the `models` folder if missing; `exist_ok=True` means it will not error if the folder is already there. This must run before `open(MODEL_PATH, "wb")`, because `open()` can create a missing *file* but not a missing *folder*.

Step 4 — app.py, Scenario 1 version (Flask's own dev server)

```

import os
import pickle

from flask import Flask, render_template, request, url_for, redirect

app = Flask(__name__)

MODEL_PATH = os.path.join("models", "gaussian_nb_model.pkl")

with open(MODEL_PATH, "rb") as f:
    model = pickle.load(f)

@app.route("/", methods=["GET"])
def index():
    return render_template("index.html")

@app.route("/predict", methods=["GET", "POST"])
def predict():
    try:

```

```

    if request.method == "GET":
        return redirect(url_for("index"))

    sepal_length = float(request.form["sepal_length"])
    sepal_width = float(request.form["sepal_width"])
    petal_length = float(request.form["petal_length"])
    petal_width = float(request.form["petal_width"])
except (KeyError, ValueError):
    return render_template(
        "result.html",
        prediction="Invalid input. Please enter valid numbers for all fields.",
        inputs=None,
    )
features = [[sepal_length, sepal_width, petal_length, petal_width]]
prediction = model.predict(features)[0]
inputs = {
    "Sepal Length": sepal_length,
    "Sepal Width": sepal_width,
    "Petal Length": petal_length,
    "Petal Width": petal_width,
}

return render_template("result.html", prediction=prediction, inputs=inputs)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)

```

Why `__name__`? Flask uses it to figure out where the project's root folder is, so it knows where to look for templates/ and static/. If you run the file directly (python app.py), `__name__` becomes `"__main__"`. If another file imports it, `__name__` becomes the module's own name instead.

Why `methods=["GET"]` on `/?` That route only shows the form — no data is being submitted, just viewed. `/predict` needs both GET (redirect back if visited directly) and POST (actual form submission).

`redirect(url_for("index"))` explained: `url_for("index")` looks up the route linked to the Python function named `index` (not the file `index.html`) and returns its path (`"/"`). Using `url_for` instead of hardcoding `"/"` means the code keeps working even if the route path is renamed later.

Step 5 — templates/index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Iris Species Predictor</title>
    <style>
        body {
            font-family: 'Segoe UI', Arial, sans-serif;
            background: #f4f6f8;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }
    </style>

```

```

        margin: 0;
    }
    .card {
        background: #fff;
        padding: 32px 40px;
        border-radius: 10px;
        box-shadow: 0 4px 16px rgba(0,0,0,0.1);
        width: 340px;
    }
</style>
</head>
<body>
    <div class="card">
        <h1>Iris Species Predictor</h1>
        <p class="subtitle">Gaussian Naive Bayes model</p>
        <form action="/predict" method="POST">
            <label for="sepal_length">Sepal Length (cm)</label>
            <input type="number" step="0.1" name="sepal_length" id="sepal_length" required>

            <label for="sepal_width">Sepal Width (cm)</label>
            <input type="number" step="0.1" name="sepal_width" id="sepal_width" required>

            <label for="petal_length">Petal Length (cm)</label>
            <input type="number" step="0.1" name="petal_length" id="petal_length" required>

            <label for="petal_width">Petal Width (cm)</label>
            <input type="number" step="0.1" name="petal_width" id="petal_width" required>

            <input type="submit" value="Predict Species">
        </form>
    </div>
</body>
</html>

```

Step 6 — templates/result.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Prediction Result</title>
</head>
<body>
    <div class="card">
        <h1>Predicted Species</h1>
        <div class="prediction">{{ prediction }}</div>

        {% if inputs %}
        <table>
            {% for key, value in inputs.items() %}
            <tr>
                <td>{{ key }}</td>
                <td>{{ value }}</td>
            </tr>
            </table>
        {% endif %}
    </div>

```

```

        </tr>
        {% endfor %}
    </table>
{% endif %}

    <a class="back-link" href="/">Try Again</a>
</div>
</body>
</html>

```

Jinja parts doing the work:

```

{{ prediction }}          -> prints the predicted species
{% if inputs %} ... {% endif %} -> only shows table if inputs exist
{% for key, value in inputs.items() %} -> loops through each field

```

Step 7 — Run and Test

```
python app.py
```

Visit:

<http://localhost:5000>

Where Does Nginx Live? (Not Used Yet)

OS	Install	Config File Location
Windows	Download zip from nginx.org, extract to C:\nginx	C:\nginx\conf\nginx.conf
Mac	brew install nginx	/opt/homebrew/etc/nginx/nginx.conf (Apple Silicon) or /usr/local/etc/nginx/nginx.conf (Intel)

Scenario 2 — Switch to Waitress

Files changed: only `app.py` — the last 2 lines.

Before (Scenario 1 — Flask’s own dev server)

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

After (Scenario 2 — Waitress)

```
from waitress import serve

if __name__ == "__main__":
    serve(app, host="0.0.0.0", port=8080)
```

	Before	After
Import	none needed	<code>from waitress import serve</code>
Server	<code>app.run(...)</code>	<code>serve(app, ...)</code>
Type	Flask dev server (not production-safe)	Waitress (production-grade WSGI server)

What Is a WSGI Server?

WSGI = Web Server Gateway Interface — a standard rulebook connecting Python web frameworks to servers.

Flask/Django (the app) <-> WSGI standard <-> Gunicorn/Waitress/uWSGI (the server)

Analogy: like a standard electrical socket — any appliance (Flask, Django) built for that socket shape works with any compatible socket (Gunicorn, Waitress).

Server	Windows	Mac	Linux
Gunicorn	No (relies on Unix-only <code>fcntl</code>)	Yes	Yes
uWSGI	Difficult/unsupported	Yes	Yes
Waitress	Yes	Yes	Yes

Why not just use `app.run()`? It is only a development server — not built for real traffic, HTTPS, or multiple simultaneous users. WSGI servers like Waitress/Gunicorn are made for production.

Nothing Else Changes

```
train_model.py -> same
index.html      -> same
result.html     -> same
```

Access

```
http://localhost:8080
http://127.0.0.1:8080
```

`localhost` is just a nickname for `127.0.0.1` — both mean “this same computer.” The OS translates one to the other automatically.

host="0.0.0.0" vs 127.0.0.1 — Not the Same Thing

host="0.0.0.0" is a **listening rule**: accept connections from anywhere (this machine, or other devices on the network). It is not an address you visit.

127.0.0.1 is the actual address of “this machine” — that’s what you type into a browser to reach it.

To reach the app from another device on the same Wi-Fi:

```
# find your LAN IP
```

```
Windows: ipconfig
```

```
Mac:      ipconfig getifaddr en0
```

```
http://192.168.1.5:8080
```

This only works on the **same local network**. A device on a different network (outside your home) cannot reach it without router port forwarding, a public domain + Nginx, or a tunnel tool like ngrok.

Private vs public IP: every device on your Wi-Fi gets its own private IP (e.g. laptop 192.168.1.5, phone 192.168.1.8) — these only work inside the home network. The whole house shares ONE public IP as seen from the internet (assigned by your ISP), and your router translates between the two (this translation is called **NAT**).

Scenario 3 — Nginx, 1 Server Block, 1 Domain, 1 app.py

Files changed: hosts file (new) + nginx.conf (new). app.py unchanged.

Scenario 2: Browser -> Waitress (port 8080) -> Flask

Scenario 3: Browser -> Nginx (port 80) -> Waitress (port 8080) -> Flask

Step 1 — hosts file

Windows: C:\Windows\System32\drivers\etc\hosts

Mac: /etc/hosts

Add:

127.0.0.1 abcd.com

Step 2 — nginx.conf

```
server {
    listen 80;
    server_name abcd.com;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

What Each proxy_set_header Line Does

Header	Tells Flask...
Host	which domain was typed
X-Real-IP	visitor's actual IP
X-Forwarded-For	full proxy chain of IPs
X-Forwarded-Proto	whether the original request was http or https

Without Host \$host, Flask would only ever see 127.0.0.1 regardless of what domain the visitor typed. Without X-Real-IP, Flask would think every request came from Nginx's own IP.

Starting and Stopping Nginx

Windows:

```
cd C:\nginx
start nginx
nginx -s stop      # immediate stop
nginx -s quit      # graceful stop
nginx -s reload    # reload config without stopping
```

Mac (Homebrew):

```
brew services start nginx
brew services stop nginx
sudo nginx -s reload
```

Full Default nginx.conf (Before Editing)

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    include mime.types;
    default_type application/octet-stream;

    sendfile on;
    keepalive_timeout 65;

    server {
        listen 80;
        server_name localhost;

        location / {
            root html;
            index index.html index.htm;
        }

        error_page 500 502 503 504 /50x.html;
        location = /50x.html {
            root html;
        }
    }
}
```

Every Line Explained

Line	Meaning
worker_processes 1;	how many worker processes Nginx runs
events { worker_connections 1024; }	each worker can handle up to 1024 simultaneous connections
include mime.types;	loads a file mapping extensions to content types
default_type application/octet-stream;	fallback type for unknown file extensions
sendfile on;	fast, direct disk-to-network file sending, skipping extra memory copies
keepalive_timeout 65;	keep a connection open 65 seconds for follow-up requests
listen 80;	which port this server block answers on
server_name localhost;	which domain(s) this block responds to
location / { root html; index index.html; }	for the default block, serve static files from the <code>html</code> folder
error_page 500 502 503 504 /50x.html;	show a custom page on server errors instead of a blank error

For this project, the default `location /` block is **replaced** with a `proxy_pass` block (shown above), since Nginx is forwarding to Waitress instead of serving static files itself.

What Did NOT Change

app.py, train_model.py, index.html, result.html -> all unchanged
port 8080 -> Waitress still runs here, untouched

Access

	Scenario 2	Scenario 3
URL	http://localhost:8080	http://abcd.com
Talks directly to	Waitress	Nginx, then Waitress
Port typed	8080	none (defaults to 80)

Scenario 4 — Nginx, 1 Server Block, 2 Domains, 1 app.py (Virtual Hosting)

Files changed: hosts, nginx.conf, and app.py.

Scenario 3: abcd.com -> Nginx -> Waitress -> Flask

Scenario 4: abcd.com, api.abcd.com -> same Nginx block, same Waitress, same Flask

Step 1 — hosts file (both domains)

```
127.0.0.1 abcd.com
127.0.0.1 api.abcd.com
```

Step 2 — nginx.conf (one block, both names)

```
server {
    listen 80;
    server_name abcd.com api.abcd.com;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
    }
}
```

Step 3 — app.py must check request.host

```
@app.route("/", methods=["GET"])
def index():
    if request.host.startswith("api."):
        return {"status": "ok", "message": "Iris Predictor API"}
    return render_template("index.html")
```

Why this file must change: Nginx forwards both domains to the same Waitress app — Flask itself has to look at the Host header to tell them apart.

What Domains and Subdomains Actually Are

A domain name is a human-friendly nickname for a computer's IP address, translated by DNS (the internet's phone book).

```
abcd.com          -> the root domain ("123 Main Street")
web.abcd.com      -> a subdomain ("123 Main Street, Suite 5")
```

Subdomains can point anywhere — same server as the root domain, or a completely different one — this is set with a DNS “A record” or “CNAME record.”

How Nginx tells abcd.com and api.abcd.com apart when both point to the same IP: the browser sends the typed name inside the Host header of the HTTP request itself:

```
GET / HTTP/1.1
Host: api.abcd.com
```

Nginx reads this header and matches it against every `server_name` line across all `server {}` blocks to decide which block handles the request.

Access

`http://abcd.com` -> shows the HTML form
`http://api.abcd.com` -> shows JSON

Scenario 5 — Nginx, 2 Server Blocks, 2 Domains, Still 1 app.py

Files changed: nginx.conf only. hosts and app.py unchanged from Scenario 4.

Before (Scenario 4 — 1 block)

```
server {
    listen 80;
    server_name abcd.com api.abcd.com;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
    }
}
```

After (Scenario 5 — 2 blocks)

```
server {
    listen 80;
    server_name abcd.com;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
    }
}

server {
    listen 80;
    server_name api.abcd.com;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
    }
}
```

Why Split If the Result Is Identical?

Each `server {}` block is fully independent. Splitting them means each domain can later get its own SSL certificate, its own `/static/` path, its own logging or rate-limiting rules — without touching the other block. In Scenario 4, any change to the shared block affects both domains at once.

`app.py` still needs the `request.host` check from Scenario 4 — Flask still cannot tell the domains apart on its own; only the Nginx routing config changed.

Access

Same URLs as Scenario 4 — behavior is identical, the difference only matters for future, independent changes per domain.

Scenario 6 — Nginx, 2 Server Blocks, 2 Waitress Instances, 2 app.py

Files changed: nginx.conf (edit), app.py (simplified), and a new file app_api.py.

Scenario 5: abcd.com, api.abcd.com -> 2 blocks -> SAME Waitress:8080

Scenario 6: abcd.com -> block A -> Waitress:8080 (app.py)
api.abcd.com -> block B -> Waitress:8081 (app_api.py)

Change 1 — app.py (remove the host check)

```
@app.route("/", methods=["GET"])
def index():
    return render_template("index.html")
```

The request.host check from Scenario 4/5 is removed — api.abcd.com no longer even reaches this file. Port stays 8080.

Change 2 — new file app_api.py

```
import os, pickle
from flask import Flask, request, jsonify
from waitress import serve

app = Flask(__name__)

with open(os.path.join("models", "gaussian_nb_model.pkl"), "rb") as f:
    model = pickle.load(f)

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    features = [[data["sepal_length"], data["sepal_width"],
                  data["petal_length"], data["petal_width"]]]
    prediction = model.predict(features)[0]
    return jsonify({"prediction": prediction})

if __name__ == "__main__":
    serve(app, host="0.0.0.0", port=8081)
```

Change 3 — nginx.conf (different ports per block)

```
server {
    listen 80;
    server_name abcd.com;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
    }
}

server {
    listen 80;
    server_name api.abcd.com;
```

```
location / {  
    proxy_pass http://127.0.0.1:8081;  
    proxy_set_header Host $host;  
}  
}
```

Run Both Apps

```
python app.py           # terminal 1, port 8080  
python app_api.py       # terminal 2, port 8081  
nginx -s reload
```

Access

```
http://abcd.com         -> app.py, HTML form  
http://api.abcd.com     -> app_api.py, JSON
```

Scenario 7 — 4 Physical Machines, Same Wi-Fi/LAN

```
laptop-1 -> browser/client
laptop-2 -> nginx
laptop-3 -> app.py      (port 8080)
laptop-4 -> app_api.py  (port 8081)
```

Step 1 — Find Each Machine's LAN IP

```
Windows: ipconfig
Mac:     ipconfig getifaddr en0
```

```
laptop-2 (nginx)      -> 192.168.1.9
laptop-3 (app.py)     -> 192.168.1.10
laptop-4 (app_api)    -> 192.168.1.11
```

Change 1 — nginx.conf on laptop-2 (real IPs, not 127.0.0.1)

```
server {
    listen 80;
    server_name abcd.com;
    location / {
        proxy_pass http://192.168.1.10:8080;
        proxy_set_header Host $host;
    }
}
server {
    listen 80;
    server_name api.abcd.com;
    location / {
        proxy_pass http://192.168.1.11:8081;
        proxy_set_header Host $host;
    }
}
```

Change 2 — hosts file moves to laptop-1 (the client)

```
192.168.1.9  abcd.com
192.168.1.9  api.abcd.com
```

Points to laptop-2's IP (nginx), not 127.0.0.1, because laptop-1 isn't running anything itself. laptop-3 and laptop-4 need no hosts entry at all — they just run their apps and wait for Nginx to forward requests.

Change 3 — 0.0.0.0 binding now matters for real

```
serve(app, host="0.0.0.0", port=8080)
```

This was already 0.0.0.0 from Scenario 2 — now critical, since laptop-2 needs to reach laptop-3 over the network. If it had been 127.0.0.1, laptop-2 could not connect at all.

Change 4 — Firewalls (new requirement)

```
laptop-3 -> allow incoming on port 8080
laptop-4 -> allow incoming on port 8081
```

laptop-2 -> allow incoming on port 80

Windows Firewall / macOS Firewall block unsolicited incoming connections by default — these must be explicitly allowed.

Scenario 8 — HTTPS via OpenSSL (Self-Signed, Browser Warns)

Files changed: nginx.conf on laptop-2 (edit) plus 2 new cert files. Nothing else changes — laptop-3/laptop-4 still talk plain HTTP to Nginx; Nginx alone handles encryption.

Step 1 — Generate a Self-Signed Certificate

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout key.pem -out cert.pem
```

Flag	Meaning
req	run OpenSSL's certificate-request tool
-x509	make a self-signed cert directly, skip the formal request step
-nodes	no password on the private key (server must start unattended)
-days 365	certificate validity: 1 year
-newkey rsa:2048	create a new 2048-bit RSA key
-keyout key.pem / -out cert.pem	where the private key / public certificate get saved

Step 2 — Add HTTPS Blocks

```
server {
    listen 443 ssl;
    server_name abcd.com;
    ssl_certificate      C:/nginx/certs/cert.pem;
    ssl_certificate_key  C:/nginx/certs/key.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        proxy_pass http://192.168.1.10:8080;
        proxy_set_header Host $host;
    }
}

server {
    listen 443 ssl;
    server_name api.abcd.com;
    ssl_certificate      C:/nginx/certs/cert.pem;
    ssl_certificate_key  C:/nginx/certs/key.pem;

    location / {
        proxy_pass http://192.168.1.11:8081;
        proxy_set_header Host $host;
    }
}
```

Optional HTTP -> HTTPS redirect:

```
server {
    listen 80;
    server_name abcd.com api.abcd.com;
    return 301 https://$host$request_uri;
}
```

The HTTP -> HTTPS Redirect Design Issue

It is tempting to put the `return 301 https://$host$request_uri;` line directly inside the same `listen 80` block that also has the `location / { proxy_pass ...; }` block for plain HTTP traffic. This is a design issue: that server block is already proxying every request through `location /`, so a `return 301` sitting after it does not cleanly express “always redirect, never proxy.”

Correct, clean setup: give the plain-HTTP redirect its own, separate `server {}` block, with no `location /` proxy at all — its only job is to send everything straight to HTTPS:

```
server {
    listen 80;
    server_name abcd.com;
    return 301 https://$host$request_uri;
}
```

`return 301` sends the browser an HTTP 301 Moved Permanently response, telling it “the resource you asked for now lives at the same host and path, but over HTTPS” — the browser then automatically makes a fresh HTTPS request.

What listen 443 ssl Does

Tells Nginx to (1) listen on port 443, the standard HTTPS port, and (2) enable TLS encryption for that port. Without the `ssl` parameter, Nginx would treat port 443 as plain HTTP, which fails because browsers expect TLS there.

ssl_protocols and ssl_ciphers

`ssl_protocols TLSv1.2 TLSv1.3;` only allows connections using these 2 modern, secure protocol versions — older protocols (SSLv3, TLSv1.0, TLSv1.1) are excluded to avoid outdated, vulnerable encryption.

`ssl_ciphers HIGH:!aNULL:!MD5;` allows only high-security ciphers, excludes ciphers with no authentication (!aNULL), and excludes the broken MD5 hash function (!MD5).

Why the Browser Still Shows “Not Secure”

Browsers only trust certificates issued by a recognized Certificate Authority (like Let’s Encrypt or DigiCert). A self-signed certificate is like a passport you printed yourself — the encryption genuinely works, but nobody official has vouched for your identity.

Fix	Good for
Click “Advanced -> Proceed anyway”	Local development only
Get a real Let’s Encrypt cert (needs a real domain, e.g. free via DuckDNS or No-IP, then Certbot)	Actual production

Getting a Publicly Trusted Certificate (Let’s Encrypt via Docker)

Let’s Encrypt requires a real domain. If you do not own one, a free dynamic DNS provider gives you a real internet hostname:

DuckDNS -> `something.duckdns.org`
No-IP -> `something.ddns.net`

Once registered, the hostname (e.g. `myflask.duckdns.org`) resolves publicly, and Let's Encrypt can work with it.

Requesting the certificate through Certbot in Docker:

```
docker run -it --rm `
-v C:\Users\CSE-17\certs:/etc/letsencrypt `
-p 80:80 `
certbot/certbot certonly --standalone -d abcdpbt.duckdns.org
```

Part	Meaning
<code>docker run</code>	starts a new container from an image
<code>-it</code>	interactive mode (<code>-i</code>) plus a pseudo-terminal (<code>-t</code>), so the console looks and behaves normally
<code>--rm</code>	automatically removes the container once it exits
<code>`</code>	PowerShell's line-continuation character
<code>-v</code>	volume mount — shares a folder between host and container;
<code>C:\Users\CSE-17\certs:/etc/letsencrypt</code>	left side is the Windows host folder, right side is the folder inside the container where Certbot writes the certificates
<code>-p 80:80</code>	port mapping, <code>host_port:container_port</code> — maps port 80 on the host to port 80 in the container, required because Let's Encrypt must be able to connect to port 80 for validation
<code>certbot/certbot</code>	the official Certbot Docker image, with Certbot pre-installed
<code>certonly</code>	Certbot subcommand meaning “only generate the certificate,” without auto-configuring Nginx/Apache — files land in <code>/etc/letsencrypt</code> for manual use
<code>--standalone</code>	Certbot spins up its own temporary web server inside the container to answer Let's Encrypt's validation request; no Nginx/Apache needs to be running on the host, but port 80 must be free
<code>-d abcdpbt.duckdns.org</code>	the domain name the certificate is being requested for

This produces `fullchain.pem` and `privkey.pem` — used the same way `cert.pem/key.pem` were used with OpenSSL.

How the validation actually works (the ACME HTTP-01 challenge): Certbot requests a certificate for the domain, and Let's Encrypt asks Certbot to prove control of that domain. Let's Encrypt generates a unique token, which Certbot makes available at a temporary URL such as `http://abcdpbt.duckdns.org/.well-known/acme-challenge/<token>`. Because `--standalone` makes Certbot run its own HTTP server on port 80 to serve that exact response, Let's Encrypt connects to the URL, retrieves it, confirms control of the domain, and issues the certificate. (If GoDaddy were the CA instead, the same basic idea applies, using GoDaddy's own domain-control validation process.)

Finding your machine's public IP (for reference):

```
(Invoke-WebRequest -Uri "https://api.ipify.org").Content
```

This asks the `api.ipify.org` service to return the public IP address the internet sees for your connection (e.g. `203.0.113.25`).

Important real-world catch: this entire flow only works if **port 80 is open from the internet** — meaning your home router must forward external port 80 to the machine running Certbot. On a typical home network behind NAT, this requires explicit router configuration, and without it the HTTP-01 challenge cannot reach your machine at all.

Scenario 9 — HTTPS via mkcert (No Warning)

Files changed: nginx.conf (edit) plus 2 new, locally-trusted cert files.

Step 1 — Install mkcert

Windows (Admin PowerShell, via Chocolatey):

```
Set-ExecutionPolicy Bypass -Scope Process -Force; `
[System.Net.ServicePointManager]::SecurityProtocol = `
[System.Net.ServicePointManager]::SecurityProtocol -bor 3072; `
iex ((New-Object System.Net.WebClient).DownloadString(`
'https://community.chocolatey.org/install.ps1'))

choco install mkcert
```

Mac:

```
brew install mkcert
```

Step 2 — Create a Local Trusted Authority

```
mkcert -install
```

This plants a private root certificate into the OS and browser trust stores. Any certificate mkcert generates afterward is automatically trusted, but **only on this machine**.

Step 3 — Generate Certs for Both Domains

```
mkcert abcd.com api.abcd.com
```

Creates:

```
abcd.com+1.pem          (certificate, covers both names)
abcd.com+1-key.pem      (private key)
```

Step 4 — nginx.conf (swap the cert paths)

```
server {
    listen 443 ssl;
    server_name abcd.com api.abcd.com;
    ssl_certificate      C:/certs/abcd.com+1.pem;
    ssl_certificate_key C:/certs/abcd.com+1-key.pem;

    location / {
        proxy_pass http://192.168.1.10:8080;
        proxy_set_header Host $host;
    }
}
```

One cert now covers both domains, so the two HTTPS blocks can even be merged back into one, following the same logic as Scenario 4.

Important Limit

This trust is local to the machine where `mkcert -install` was run (laptop-2). If laptop-1 (the browser/client) is a different machine, it will not automatically trust this certificate either — mkcert's trust does not travel

across machines the way a real CA's trust does.

Scenario 10 — Serving Static Files via Nginx

Files changed: nginx.conf (edit), a new static/ folder, and index.html / result.html (edit to link external CSS).

Scenario 9: ALL requests -> Nginx -> Flask/Waitress (even CSS, images)

Scenario 10: /static/* -> Nginx handles directly (fast)
everything else -> Nginx -> Flask/Waitress (as before)

Step 1 — Create a static/ Folder

```
mlops2/
|-- app.py
|-- templates/
|   |-- index.html
|   |-- result.html
|-- static/
|   |-- style.css
```

Step 2 — Move CSS Out of the HTML Files

Before (index.html, inline <style>):

```
<head>
  <style>
    body { font-family: 'Segoe UI', Arial, sans-serif; }
  </style>
</head>
```

After (index.html, external link):

```
<head>
  <link rel="stylesheet" href="{ url_for('static', filename='style.css') }">
</head>
```

url_for('static', filename=...) is a built-in Flask helper that always resolves to /static/style.css without manual typing. The same change applies to result.html.

Step 3 — nginx.conf (add a /static/ block)

```
server {
    listen 443 ssl;
    server_name abcd.com api.abcd.com;
    ssl_certificate      C:/certs/abcd.com+1.pem;
    ssl_certificate_key  C:/certs/abcd.com+1-key.pem;

    location /static/ {
        root C:/mlops2;
    }

    location / {
        proxy_pass http://192.168.1.10:8080;
        proxy_set_header Host $host;
    }
}
```

Path resolution: a request for `abcd.com/static/style.css` makes Nginx look for the file at `C:/mlops2/static/style.css`. `root` points to the project folder, not the `static` folder itself — Nginx appends the rest of the URL automatically.

Why Web Servers Serve Static Files Faster

1. **Efficient file I/O:** Nginx uses `sendfile()` to send files directly from disk to the network socket inside the kernel, with no copy into Nginx's own memory and no extra context switches.
2. **No app logic overhead:** Nginx just maps URLs to files — no routing, no Python, no processing.
3. **Event-driven and asynchronous:** Nginx handles thousands of requests with very low resource usage.
4. **Built-in compression and caching:** gzip/Brotli compression and headers like `ETag/Cache-Control` let the browser reuse a previously downloaded file instead of re-requesting it (a `304 Not Modified` response sends no file at all if nothing changed).

Application servers (Flask + Waitress) must load Python code, parse the request, read files via Python, and return the response manually — this makes them CPU-heavier and slower for static files by comparison.

Important Note on Separate HTTP/HTTPS Blocks

Since each `server {}` block is independent, if you also kept a plain HTTP (port 80) server block, the `location /static/` block must be added to **both** the HTTP and HTTPS blocks, or static files will break on whichever one you forgot.

Scenario 11 — Load Balancing and Sticky Sessions

Files changed: `nginx.conf` (edit, new `upstream` block). No new application files — instead, multiple copies of `app.py` are run.

Scenario 10: abcd.com -> Nginx -> ONE app.py (port 8080)

```
Scenario 11:  abcd.com -> Nginx -> upstream pool -> app.py #1 (8080)
               -> app.py #2 (8082)
               -> app.py #3 (8083)
```

Step 1 — Run Multiple Copies, Different Ports

```
serve(app, host="0.0.0.0", port=8080) # copy 1
serve(app, host="0.0.0.0", port=8082) # copy 2
serve(app, host="0.0.0.0", port=8083) # copy 3
```

```
python app.py           # terminal 1, port 8080
python app2.py          # terminal 2, port 8082
python app3.py          # terminal 3, port 8083
```

Step 2 — nginx.conf (add upstream, change proxy_pass)

```
upstream abcd_servers {
    server 192.168.1.10:8080;
    server 192.168.1.10:8082;
    server 192.168.1.10:8083;
}

server {
    listen 443 ssl;
    server_name abcd.com;
    ssl_certificate      C:/certs/abcd.com+1.pem;
    ssl_certificate_key  C:/certs/abcd.com+1-key.pem;

    location /static/ {
        root C:/mlops2;
    }

    location / {
        proxy_pass http://abcd_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Default Behavior: Round-Robin

No extra setting is needed — Nginx cycles through the list automatically:

```
Request 1 -> 8080
Request 2 -> 8082
Request 3 -> 8083
Request 4 -> 8080 (cycles back)
```

Other Load Balancing Algorithms

```
upstream abcd_servers {
    least_conn;                # fewest active connections gets the next request
    server 192.168.1.10:8080;
    server 192.168.1.10:8082;
    server 192.168.1.10:8083;
}
```

```
upstream abcd_servers {
    ip_hash;                   # same client IP always -> same server
    server 192.168.1.10:8080;
    server 192.168.1.10:8082;
    server 192.168.1.10:8083;
}
```

For load balancing across multiple physical/virtual machines instead of ports on one machine, the same idea applies with different IPs:

```
upstream waitress_servers {
    server 10.0.2.45:8000;
    server 10.0.2.46:8000;
    server 10.0.2.47:8000;
}
```

`proxy_set_header X-Real-IP $remote_addr;` passes the actual client IP to the app — without it, every request appears to come from Nginx's own IP. `proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;` passes the full chain of proxy IPs, useful if there is more than one proxy in the path.

Sticky Sessions (Nginx Plus Only, Not in Free Nginx)

The simplest form just turns stickiness on, with Nginx Plus picking a default tracking method:

```
upstream abcd_servers {
    sticky;
    server 192.168.1.10:8080;
    server 192.168.1.10:8082;
    server 192.168.1.10:8083;
}
```

The more explicit, commonly used form names the exact cookie and combines it with a load-balancing algorithm:

```
upstream abcd_servers {
    least_conn;
    sticky cookie srv_id expires=1h path=/;
    server 192.168.1.10:8080;
    server 192.168.1.10:8082;
    server 192.168.1.10:8083;
}
```

Stickiness can be tracked per user in a few different ways: by client IP (the `ip_hash` method), by a session cookie (the `cookie` method shown above), or by a custom token (used by application-aware load balancers).

Piece	What it does
<code>sticky cookie srv_id</code> <code>expires=1h</code> <code>path=</code>	Nginx sets a cookie recording which server the visitor is assigned to that assignment expires after 1 hour of inactivity the cookie applies site-wide
If that server goes down	Nginx automatically reassigns the visitor to a healthy one

Example: User A's first request lands on server 8082. A cookie is set. All of User A's future requests, for the next hour, are routed back to 8082. User B, first landing on 8080, stays on 8080.

For this Iris predictor specifically, each prediction request is fully self-contained — no memory of past requests is needed — so plain round-robin or `ip_hash` is sufficient. Sticky sessions matter far more for apps with logins, shopping carts, or multi-step flows.

Appendix — Quick Reference

The Full Ladder, One Table

#	What's New	Key Concept
1	Train + run with Flask's own server	<code>app.run()</code>
2	Switch to Waitress	WSGI production server
3	Nginx in front of Waitress	<code>proxy_pass</code> , reverse proxy
4	1 domain-agnostic Nginx block	<code>server_name domain1 domain2;</code> , <code>request.host</code> in Flask
5	Split into independent blocks	Each <code>server {}</code> is isolated config
6	Different apps per domain	Different <code>proxy_pass</code> ports, 2 Flask files
7	Real network, 4 machines	Real IPs instead of 127.0.0.1, firewall, 0.0.0.0 binding
8	HTTPS (self-signed)	<code>openssl</code> , <code>ssl_certificate</code> , trusted-CA gap
9	HTTPS (locally trusted)	<code>mkcert</code> , local root CA, dev-only
10	Static file offloading	<code>location /static/</code> , <code>sendfile()</code> efficiency
11	Multiple app copies	<code>upstream</code> , round-robin / <code>least_conn</code> / <code>ip_hash</code> , sticky cookies

Ports Used Throughout

Port	Used By
5000	Flask dev server (Scenario 1 only)
8080	Waitress running <code>app.py</code>
8081	Waitress running <code>app_api.py</code> (Scenario 6+)
8082, 8083	Extra <code>app.py</code> copies for load balancing (Scenario 11)
80	Nginx, plain HTTP
443	Nginx, HTTPS

Files That Ever Change Across All 11 Scenarios

<code>app.py</code>	-> server type (1-2), host check added (4-5), host check removed (6)
<code>app_api.py</code>	-> new in Scenario 6
<code>train_model.py</code>	-> never changes after Scenario 1
<code>index.html</code>	-> unchanged until Scenario 10 (external CSS link)
<code>result.html</code>	-> unchanged until Scenario 10 (external CSS link)
hosts file	-> new in Scenario 3, grows in Scenario 4, moves machine in Scenario 7
<code>nginx.conf</code>	-> new in Scenario 3, edited in every scenario from 4 onward
<code>cert.pem</code> / <code>key.pem</code>	-> new in Scenario 8 (<code>openssl</code>)
<code>abcd.com+1.pem</code> / <code>-key.pem</code>	-> new in Scenario 9 (<code>mkcert</code>), replaces Scenario 8's certs
<code>static/style.css</code>	-> new in Scenario 10